

LAB-1

* Tic-tac-toe

~~def check_pos(r, c):~~
~~if 0 <= r < 3 and 0 <= c < 3:~~

function main()

initialize board as 3x3 array

Set current player to 'X'

while true:

get move (row, column) from current player
display board

if valid move:

Update board

else

if current player wins:

ask again

display winner

break

else

ask again

if Count == 9

its a draw

call main()

[Signature]

1/12

* Vacuum Cleaner Agent

```
function vacuumCleaner (environment):
```

```
    pos = (0,0)
```

```
    cleaned_cells_count = 0
```

```
    while true:
```

```
        if environment[pos] is dirty:
```

```
            clean environment[pos]
```

```
            cleaned_cells_count += 1
```

```
            print("Cleaned position:", pos)
```

```
            next_position = findNextDirty(environment)
```

```
            if next_position exists:
```

```
                position = next_position
```

```
            else: print
```

```
                print("No more dirty cells found,  
                    cleaning complete")
```

```
                break
```

```
function findNext Dirty(environment):
```

```
    for each cell in environment
```

```
        if cell is dirty
```

```
            return cell's position
```

```
    return None.
```


*

function: BFS (start state, goal state)

$\text{queue} = [(\text{start_state}, [])]$

↓

$(\text{starting \& state})$

↘ (path)

visited = { }

while queue is not empty:

current state, path = dequeue (queue)

if current state = goal state
return path

visited.add(wordend - state)

for each valid move:

new state = apply move (undo old take, money)

if new state is not in visited
enqueue (queue, (new state, path + [move]))

Return None

* Iterative deepening search algorithm *

Function depth-limited-search(node, goal, depth)

if depth is 0 and node is goal:

Return node

else if depth > 0

for each neighbour of node:

result = Depth-limited-search

(neighbour, goal, depth-1)

if result is not None:

Return Result

Return None

Function Iterative-Deepening-search(start, goal, max-depth)

for depth from 0 to max-depth

print "Searching at depth: depth"

result = Depth-limited-search(start, goal, depth)

if result is Not None:

Return Result

Return None

Function Get-Neighbours(node):

Return list of neighbours of the node

// Main Program

start_Node = 'A'

goal_Node = 'G'

max_depth = 3

result = Iterative_deepening_search(start_node, goal_node, max_depth)

if Result:

print "Goal found!"

Else

print "Goal not found within depth limit"

⇒ Output:

Searching at depth: 0

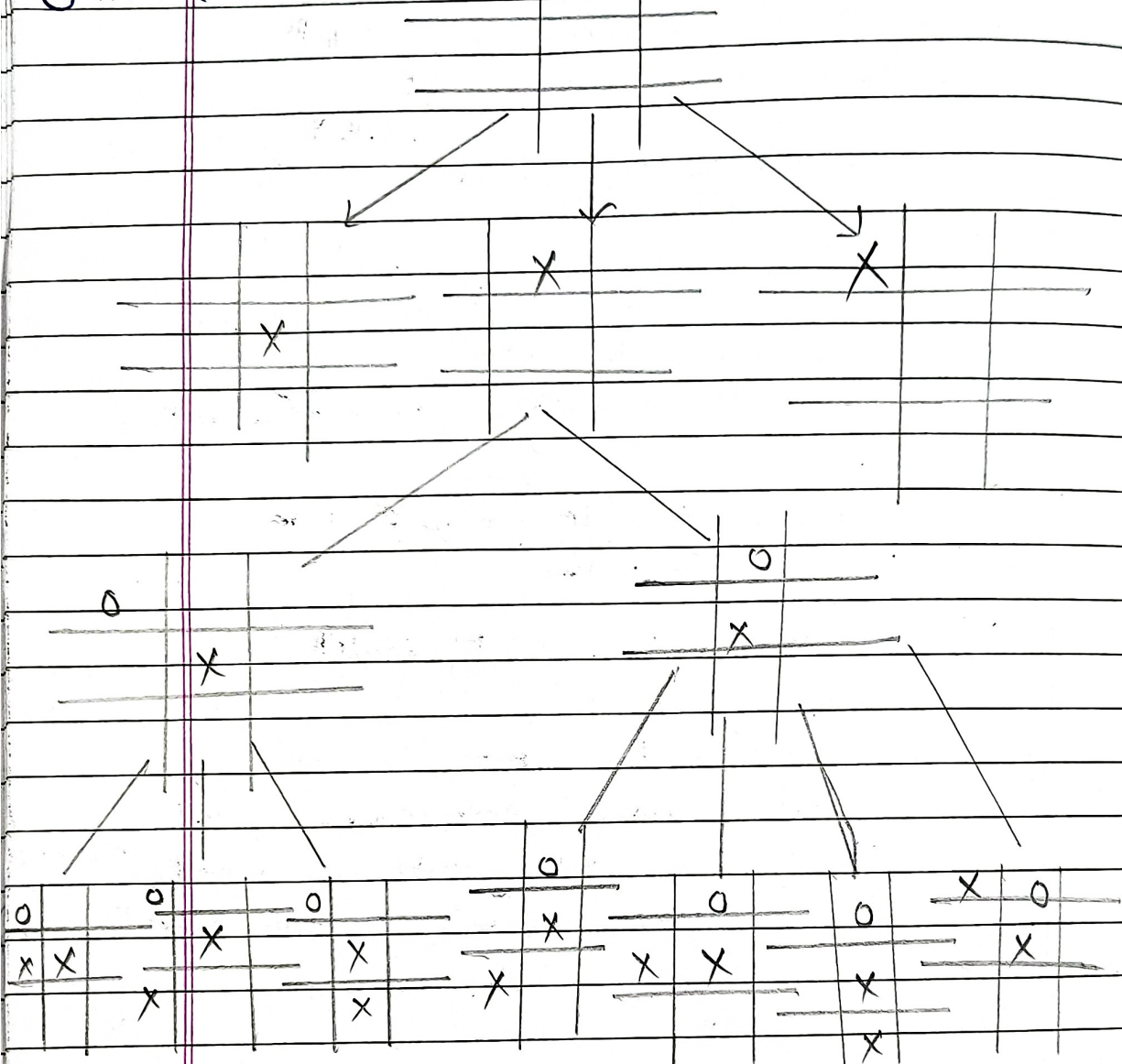
Searching at depth: 1

Searching at depth: 2

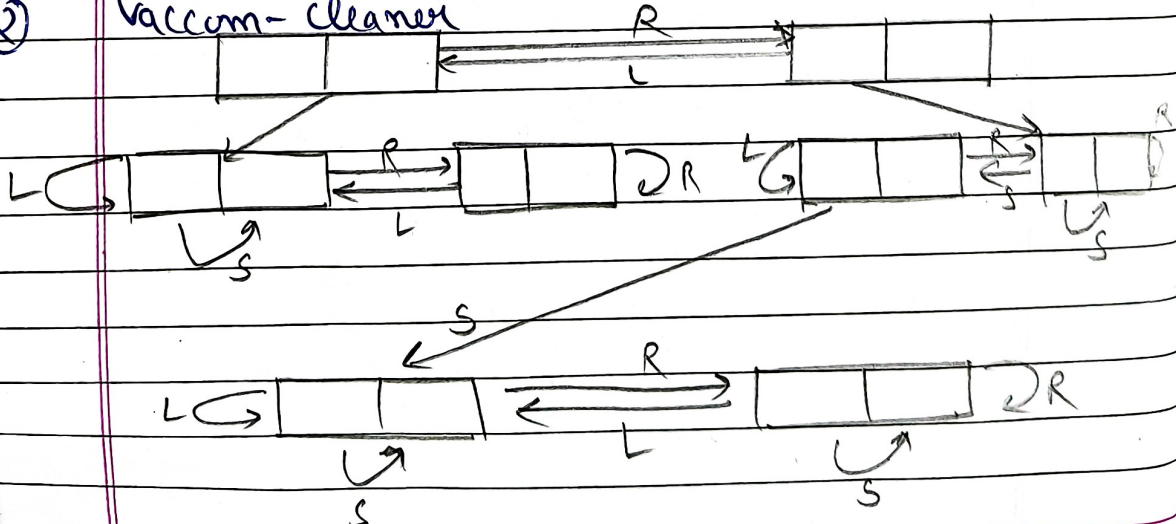
Goal G found.

state space

① Tic-Tac-Toe



② vacuum-cleaner



* State-space tree (8-puzzle problem)

1	2	3
8	0	4
7	6	5

1	0	3
8	2	4
7	6	5

1	2	3
8	4	0
7	6	5

1	2	3
8	6	4
7	0	5

1	2	3
0	8	4
7	6	5

0	1	3
8	2	4
7	6	5

1	3	0
8	2	4
7	6	5

1	2	0
8	4	3
7	6	5

1	2	3
8	4	5
7	6	0

1	2	3
8	6	4
7	0	5

1	2	3
0	8	4
7	6	5

0	2	3
8	6	4
7	0	5

8	1	3
0	2	4
7	6	5

1	3	4
8	2	0
7	6	5

1	0	2
8	4	3
7	6	5

1	2	3
8	4	5
7	0	6

1	2	3
8	6	0
7	0	5

2	0	3
1	8	4
7	6	5